
갤-러그 스타일
시스템프로그래밍및실습 8조
PvP 비행기 슈팅 게임
최-종 발표

목차

1. 프로젝트 개요 – Galaga(ギャラガ)
2. 하드웨어 설계 및 구조
3. 소프트웨어 설계 및 구조
4. 변경 사항 및 어필 사항
5. Demo

차례

1 프로젝트 개요 – Galaga(ギャラガ)

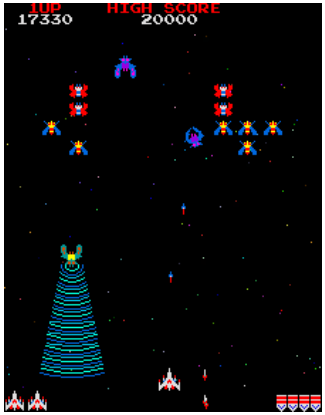
2 하드웨어 설계 및 구조

3 소프트웨어 설계 및 구조

4 변경 사항 및 어필 사항

5 Demo

프로젝트 개요 – Galaga(ギャラガ)



Motivation – Galaga(ギャラガ)

- ✓ 우주 공간에 날아다니는 적들을 쏴 죽이는 게임
- ✓ 재밌습니다.

주제 – PvP 기반 비행기 슈팅 게임

- ✓ 각 플레이어는 다음의 역할을 수행합니다.
  - – {총 발사, 궁극기, 방어막 사용}
  - – 비행기를 좌우로 컨트롤합니다.
- ✓ 상대방의 체력을 모두 소진시키는 플레이어가 승리합니다.

차례

1 프로젝트 개요 – Galaga(ギャラガ)

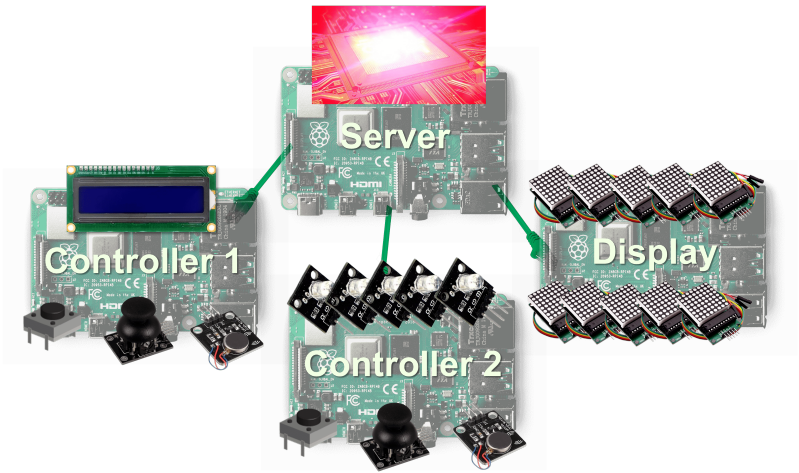
2 하드웨어 설계 및 구조

3 소프트웨어 설계 및 구조

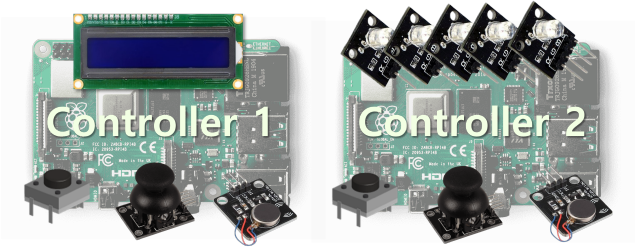
4 변경 사항 및 어필 사항

5 Demo

하드웨어 설계 및 구조



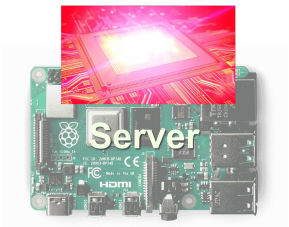
하드웨어 설계 및 구조– Controller 1,2

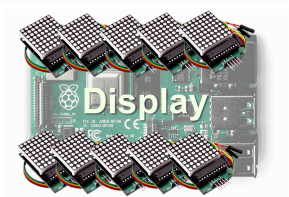


Sensor(Controller 1 ∪ 2)

- ✓ Joystick — 비행기 방향 조종
- ✓ Button($\times 2$) — 궁극기 발동
- ✓ Haptic — HP 감소 알림
- ✓ LCD W/O I2C — CTRLR 1의 게임 상태
- ✓ LED($\times 5$) — CTRLR 2의 게임 상태

하드웨어 설계 및 구조– Server, Display





Sensor(Server ∪ Display)

- ✓ 64×32 Dot Matrix 게임 화면

차례

1 프로젝트 개요 – Galaga(ギャラガ)

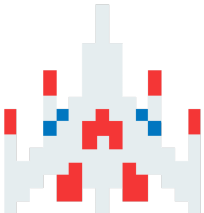
2 하드웨어 설계 및 구조

3 소프트웨어 설계 및 구조

4 변경 사항 및 어필 사항

5 Demo

소프트웨어 설계 및 구조– Game Object(게임 객체)



비행기(플레이어 1,2)

- ✓ 비행기 ⟺ { 체력, 위치, 모양 }

총알, 궁극기(*Phoenix*), 방어막

- ✓ 총알 ⟺ { 속도(속력 + 방향), 데미지, 연사량}
- ✓ 궁극기 ⟺ { 쿨타임, 데미지, 기능(총알 없애기)}
- ✓ 방어막 ⟺ { 기능(데미지 무시), 지속 시간, 쿨타임}

이팩트

- ✓ 궁극기나 플레이어와 만날 경우, 총알 사라지는 이팩트

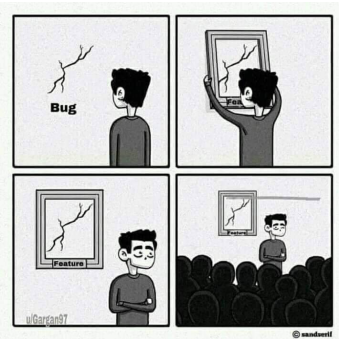
소프트웨어 설계 및 구조– Game Logic(게임 로직)

개요

- ✓ `Update` ⇒ `Rendering` ⇒ `Update` . . .

`Update` – 1 frame per 1000 ms

- ✓ (최적화) 충돌 판정
  - 비행기 – 총알, 비행기 – 궁극기,
  - 총알 – 궁극기, 총알 – 방어막,
  - 방어막 – 궁극기



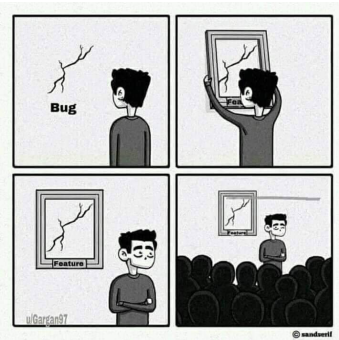
소프트웨어 설계 및 구조– Game Logic(게임 로직)

개요

- ✓ `Update` ⇒ `Rendering` ⇒ `Update` . . .

`Update` – 1 frame per 1000 ms

- ✓ (최적화) 객체 관리
  - 플레이어 컨트롤에 따른 상태 업데이트
    - {체력, 쉴드, 궁극기 사용 여부, 방어막, 총알 모드}
  - 재사용
    - {총알, 충돌 이팩트}



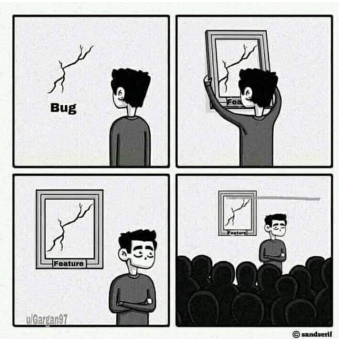
소프트웨어 설계 및 구조– Game Logic(게임 로직)

Rendering

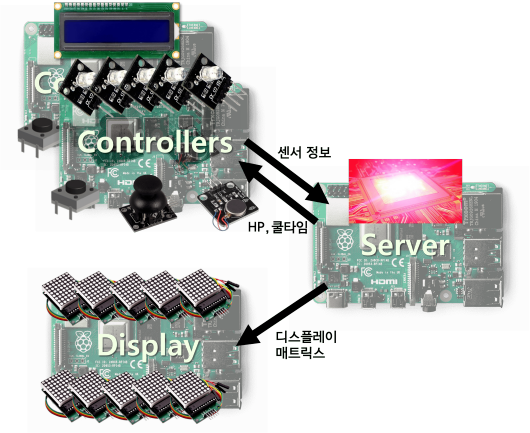
- ✓ 소켓 통신을 통해 디스플레이로 게임 상태 전송
  - State : 2D `Matrix`에 정보 압축 후 전송
  - (최적화) 변화하는 부분만 디스플레이 상태 업데이트

Controller Logic

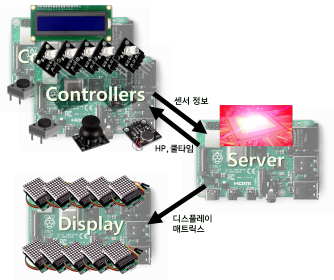
- ✓ 서버로부터 각 플레이어 상태 수신({체력, 쿨타임})
  → {LCD w/o, LEDs}에 출력



소프트웨어 설계 및 구조– Socket Logic



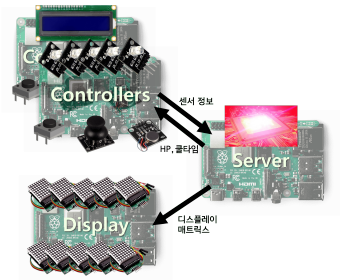
소프트웨어 설계 및 구조– Socket Logic



서버가 하는 일 – Using Socket

- ✓ ⇒ 디스플레이
  - 디스플레이가 소켓 오픈 → 서버 접속 → 출력 매트릭스
- ✓ ⟺ 컨트롤러1,2
  - 서버 파이가 소켓 오픈 → 컨트롤러 접속
  - ⇒ 센서 정보(Joystick, Button) 서버에 전달
  - ⇐ 플레이어 정보(HP, 쿨타임) 플레이어에 전달

소프트웨어 설계 및 구조– Socket Logic



서버가 하는 일 – 게임 정보 연산 후 업데이트

- ✓ 플레이어 객체 정보
  - HP, 총알 맞음, 위치 등
- ✓ 총, 궁극기, 방어막 객체 정보
  - 총 모드, { 궁극기, 방어막} 쿨타임

차례

1 프로젝트 개요 – Galaga(ギャラガ)

2 하드웨어 설계 및 구조

3 소프트웨어 설계 및 구조

4 변경 사항 및 어필 사항

5 Demo

변경 사항 및 어필 사항– *Problems and Sovled!*

Technical issue

✓ ~~너무 많은 문제를 맞닥뜨리고 해결했으나 여백이 부족한 관계로..~~

1. Synchronization : 게임 화면과 게임 상태의 동기화 문제
   Socket에서 정보 전송 시 (송신 주기 $<$ Read 주기), buffer에 누적되어 packet이 too old.
   - buffer write 시 flush가 시행되어야 함.
2. Frame : 게임의 속도가 CPU의 연산량에 따라 바뀌는 문제
   - $frame =$ Cycle per second
   - $clock \approx 10 \times \max_{1 \le i \le 10^6} (\text{Time per Cycle}_i)$
   - $frame = 1000$으로 고정, $clock$을 게임의 한 cycle로 고정 $\rightarrow$ 게임의 속도를 일정하게 유지
   - 한 번의 $clock$에 $\approx 50\mu s$정도 소요되도록 최적화

변경 사항 및 어필 사항– *Problems and Sovled!*

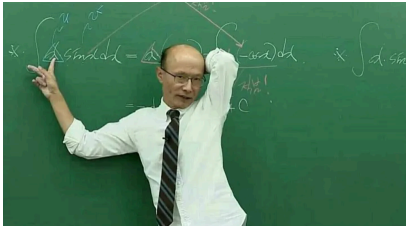
Synchronous to Asynchronous

1. Socket : 동기(Synchronous)방식 사용 시, 특정 Socket을 위해 과도한 blocking 발생
   - pthread 사용 → thread 분리
   - 다른 job과 독립적으로 시행(= 비동기, Asynchronous)

Optimization

- ✓ Resource management
  - GUI update — sleep( ) 이용하여 차이를 인지할 수 없는 수준에서 연산량 최소화
  - Object(이팩트, 총알) — 사용되고 사라진 object 재사용 → heap 영역 사용량 최소화
- ✓ Self recovery — 4 pi 중, 문제가 발생한 pi 만 복구하면 그대로 게임 진행 가능
- ✓ Display — Double buffering 사용, 상태의 Δ 만 디스플레이에 반영

변경 사항 및 어필 사항– *Appeal*



어필 사항 (고생)

- ~~너무 많은 고통을 받았으나 여백이 부족한 관계로..~~
- 64×32 도트 LED 매트릭스 연결
- 기타 Sensor 연결
- SOCKET
- 복잡한 게임 로직과 최적화 ($3\,000$ 줄의 방대한 코드)

차례

1 프로젝트 개요 – Galaga(ギャラガ)

2 하드웨어 설계 및 구조

3 소프트웨어 설계 및 구조

4 변경 사항 및 어필 사항

5 Demo

Enjoy